

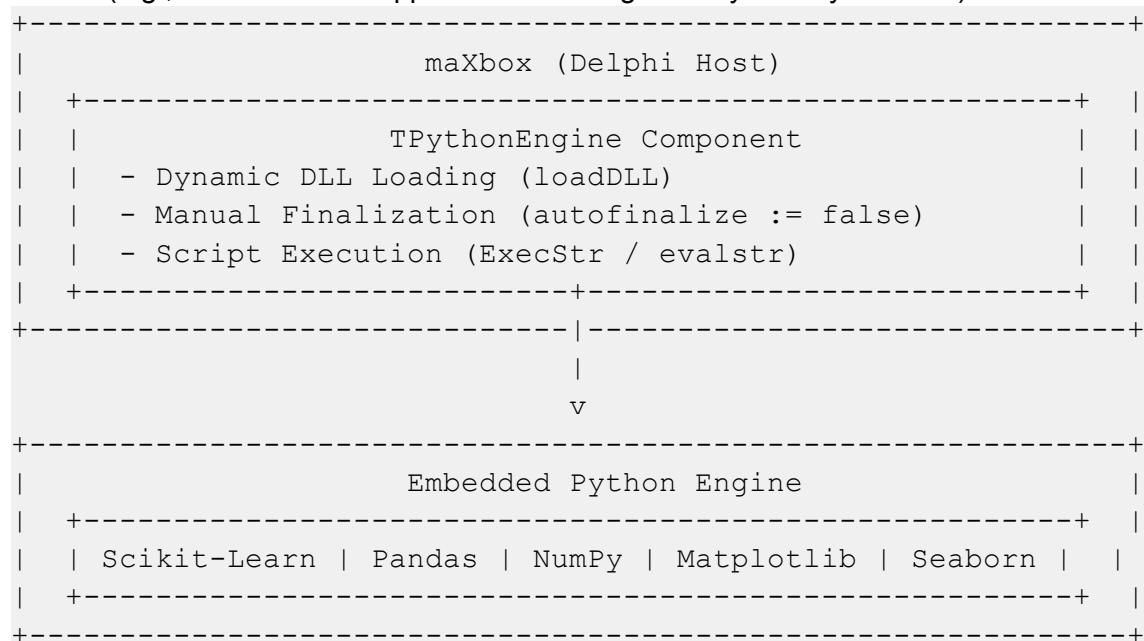
Supervised Machine Learning Workflows in maXbox: A Comprehensive Study Guide for maXbox Starter Tutorial 200

1. Executive Overview & Environment Architecture

This study guide details the execution of supervised machine learning workflows within the **maXbox** environment (specifically maXbox5 and future maXbox6). The architecture leverages **Python4Delphi (P4D)** to embed Python language execution into a native Delphi container application.

Core Architectural Concepts

- **Host Application Container** : Delphi / maXbox acts as the host framework controlling script runtime and memory lifecycle.
- **Bridge Component** : TPythonEngine dynamically loads the native Python dynamic-link library (loadDLL), enabling execution of Python scripts directly inside formatted Pascal code blocks.
- **Python Target Runtimes** : Compatible with Python versions such as 3.13 and 3.14 (e.g., C:\Users\User\AppData\Local\Programs\Python\Python314\).



Required Prerequisites & Installation

To execute the data science and machine learning workflows, the underlying Python environment requires specific core libraries. These can be installed via pip:

```
pip install pandas numpy matplotlib seaborn scipy scikit-learn
statsmodels sys os
```

2. Dataset Analysis: The Iris Flower Dataset

The workflow utilizes the **Iris dataset** (imported from `sklearn.datasets`), a classic multiclass classification dataset widely used for benchmark testing and machine learning instruction.

Dataset Specifications

1. **Total Sample Size** : 150 instances (50 samples per class).
2. **Target Classes (Species)** :
3. Setosa (Class Index 0)
4. Versicolor (Class Index 1)
5. Virginica (Class Index 2)
6. **Feature Dimensions** : 4 numeric continuous attributes measured in centimeters:
7. **Sepal Length** : sepal length (cm)
8. **Sepal Width** : sepal width (cm)
9. **Petal Length** : petal length (cm)
10. **Petal Width** : petal width (cm)

Dataset Exploration & Visualization Workflow

Data in raw array formats must be structured into a standard DataFrame using pandas to facilitate exploration and feature engineering.

Exploratory Python Script Pattern:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets

# Load raw dataset and convert to DataFrame
irisd = datasets.load_iris()
irisdf = pd.DataFrame(
    data=np.c_[irisd['data'], irisd['target']],
    columns=irisd['feature_names'] + ['target']
)

# Map target numeric identifiers to species names
species = []
for i in range(len(irisd['target'])):
    if irisd['target'][i] == 0:
        species.append('setosa')
    elif irisd['target'][i] == 1:
        species.append('versicolor')
    else:
        species.append('virginica')

irisdf['species'] = species
```

```

# Verify class distribution
print(irisdf.groupby('species').size())

# Generate scatter plot comparing Petal Length vs. Petal Width
setosa = irisdf[irisdf.species == 'setosa']
versicolor = irisdf[irisdf.species == 'versicolor']
virginica = irisdf[irisdf.species == "virginica"]

fig, ax = plt.subplots()
fig.set_size_inches(10, 5)

ax.scatter(setosa["petal length (cm)"], setosa["petal width (cm)"],
label="Setosa", facecolor="blue")
ax.scatter(versicolor["petal length (cm)"], versicolor["petal width
(cm)"], label="Versicolor", facecolor="green")
ax.scatter(virginica["petal length (cm)"], virginica["petal width
(cm)"], label="Virginica", facecolor="red")

ax.set_xlabel("petal length (cm)")
ax.set_ylabel("petal width (cm)")
ax.grid()
ax.set_title("Iris petals maXbox5")
ax.legend()
plt.show()

```

3. Supervised Learning Pipeline Architecture

The supervised machine learning pipeline follows a 9-step structured execution model controlled within maXbox.

```

[1. Create TPythonEngine] ---> [2. Load Python DLL] ---> [3. Import
Libraries]
|
[6. Train RandomForest] <--- [5. Split Train/Test] <--- [4. Load &
Structure Data]
|
v
[7. Evaluate Performance] ---> [8. Plot Confusion Matrix] ---> [9.
Free Resources]

```

The 9 Execution Steps

1. **Engine Initialization** : Instantiate the engine using `TPythonEngine.Create(Nil)` and dynamically bind to Python binaries via `loadDLL`.

2. **Lifecycle Control** : Set autofinalize := false to manually control when the Python runtime context terminates.
3. **Library Ingestion** : Execute ExecStr commands to import key packages (pandas, matplotlib.pyplot, seaborn, sklearn.model_selection.train_test_split, sklearn.ensemble.RandomForestClassifier, sklearn.metrics.accuracy_score, sklearn.metrics.confusion_matrix).
4. **Data Transformation** : Load datasets.load_iris() and map features into a pandas.DataFrame.
5. **Stratified Data Splitting** : Partition features (\$X\$) and target labels (\$y\$) into training and testing sets using train_test_split.
6. **Model Training** : Initialize a RandomForestClassifier and fit it using the training partition.
7. **Model Evaluation** : Generate predictions on test features, calculate classification accuracy using accuracy_score, and evaluate detailed class performance.
8. **Visualization** : Construct a confusion matrix DataFrame, annotate it using seaborn.heatmap(), and render with matplotlib.pyplot.show().
9. **Resource Finalization & Error Handling** : Capture execution errors in Delphi using try...except raiseError constructs and release engine memory via finally Free.

4. Model Training & Parameter Configurations

Dataset Partitioning Parameters

Data partitioning is managed through train_test_split to prevent data leakage and ensure representative target class distributions across sets:

- **Test Size (test_size=0.5)** : 50% of data reserved for testing (75 samples); 50% reserved for training (75 samples).
- **Stratification (stratify=iris.target)** : Ensures balanced class ratios across both training and testing datasets (25 samples per class in both sets).
- **Random State Seed (random_state=123456)** : Guarantees reproducibility across test runs.

Classifier Model Hyperparameters

The classification model selected is **Random Forest Classifier**

(sklearn.ensemble.RandomForestClassifier), an ensemble learning algorithm.

- **n_estimators=100** : Constructs 100 individual decision trees in the forest.
- **oob_score=True** : Calculates Out-of-Bag generalization performance.
- **random_state=123456** : Ensures deterministic tree building for consistent comparison.

5. Model Evaluation & Performance Metrics

Evaluating classifier effectiveness relies on three key primary metrics: **Accuracy** , **Precision** , and **Recall** .

Core Definitions

- **Precision** : The accuracy of positive predictions. Measures how many retrieved positive predictions were actually relevant.
$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$
- **Intuitive Rule** : If you recognize a person for someone else (confuse them with another), your classifier suffers from **bad precision**.
- **Recall** : The ratio of positive instances that are correctly detected by the classifier.
$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$
- **Intuitive Rule** : If you fail to recognize a person at all, your classifier suffers from **bad recall**.

Detailed Performance Breakdown

Classification Report Data

Upon running `print(metrics.classification_report(y_test, rf.predict(X_test), digits=3))`, the model generates the following output:

	Class / Metric	Class Name	Precision	Recall	F1-Score	Support
	-----	-----	-----	-----	-----	-----
0	Setosa	1.000	1.000	1.000	25	
1	Versicolor	0.833	1.000	0.909	25	
2	Virginica	1.000	0.800	0.889	25	
	Overall Accuracy		0.933	75		
	Macro Average		0.944	0.933	0.933	75
	Weighted Average		0.944	0.933	0.933	75

Overall Classification Accuracy : 0.9333333333333333 (93.33%). Total misclassifications: 5 out of 75 test instances .

precision recall f1-score support eg: Setosa(0), Versicolor(1), Virginica(2)

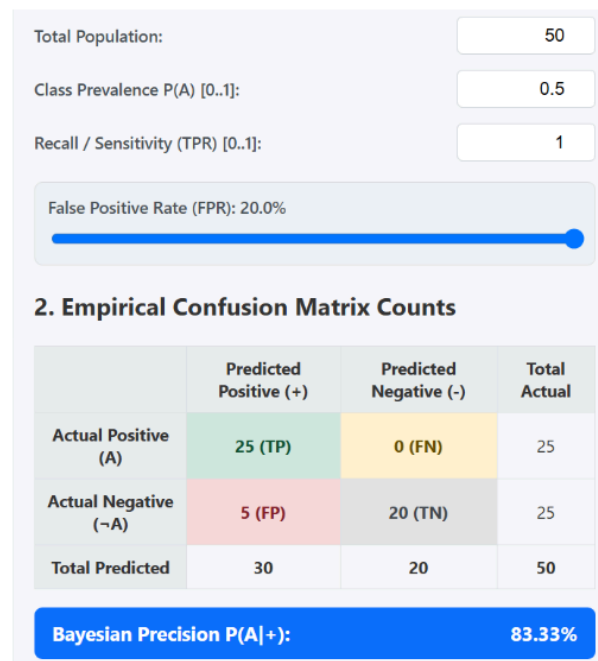
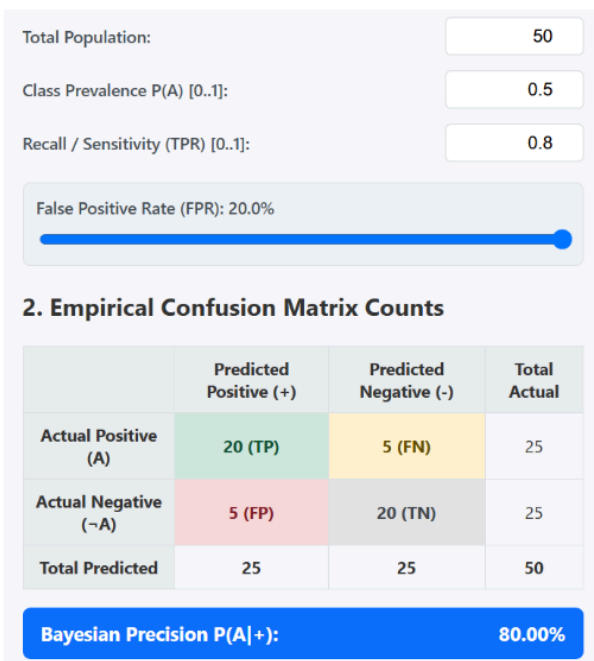
1		precision	recall	f1-score	support
2	0	1.000	1.000	1.000	25
3	1	0.833	1.000	0.909	25
4	2	1.000	0.800	0.889	25
5					
6	accuracy			0.933	75

Analysis of Misclassifications

- **Virginica Recall Analysis (Class 2)** :
- **Recall Score** : 0.800 (80%).
- **Analysis** : Out of 25 actual Virginica samples, 80% (20 samples) were correctly recognized. The remaining 20% (**5 samples**) were misclassified by the model as *Versicolor*.

- **Versicolor Precision Analysis (Class 1) :**
- **Precision Score :** 0.833 (83.33%).
- **Analysis :** The model predicted a total of 30 samples as Versicolor (25 true Versicolor samples + 5 misclassified Virginica samples).
- **Calculation :** $\frac{25}{30} = 0.8333$ (83.33% precision).
- **Setosa Accuracy (Class 0) :**
- Class 0 demonstrated perfect separation with 1.000 Precision and 1.000 Recall across all 25 test samples.

Evaluating a classifier involves three primary performance measures: the **confusion matrix**, **recall**, and **precision**.



1. Confusion Matrix

- **What it measures:** A grid that compares actual target classes against predicted classes to show exactly how many instances were correctly classified versus misclassified.
- **How to interpret it:** The diagonal cells represent correct predictions, while off-diagonal cells highlight where the model confuses one class for another (false positives and false negatives).

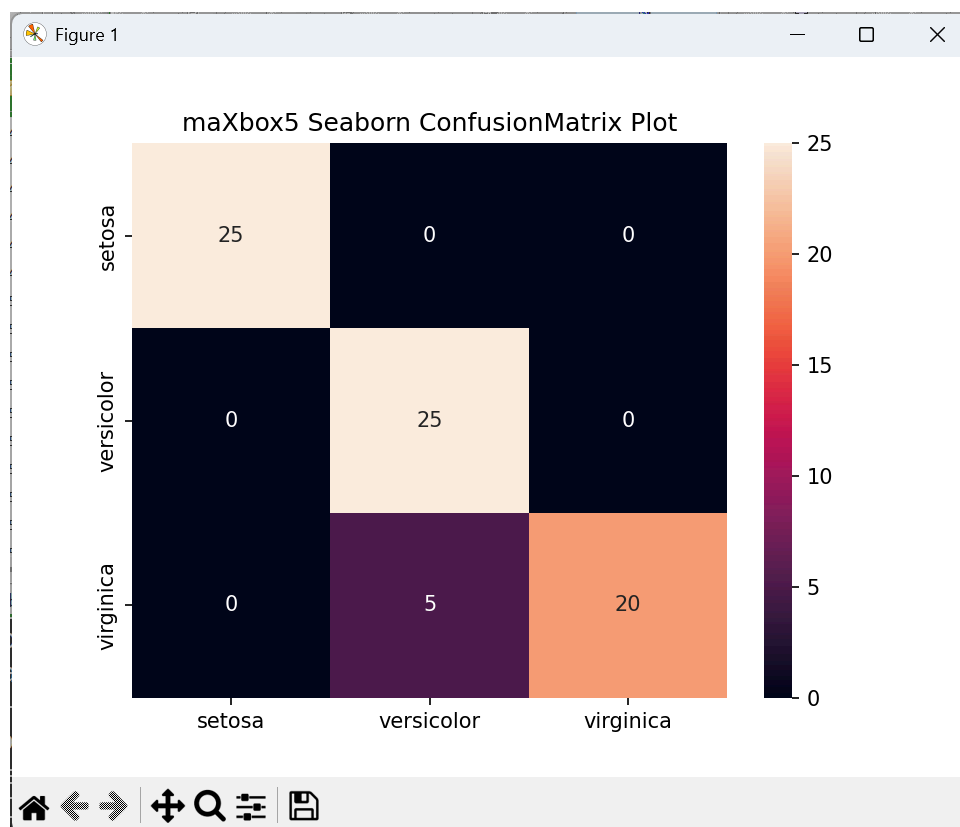
2. Recall (Sensitivity)

- **What it measures:** The ratio of actual positive instances that are truly detected by the classifier.
- **Practical rule:** **Bad recall means failing recognize something that is actually there.**
- **Example:** In the Iris classification model, a **Virginica recall of 80%** means 80% of true Virginica flowers were correctly detected, while **20% (5 samples) were missed** and misclassified as Versicolor.

3. Precision (Positive Predictive Value)

- **What it measures:** The accuracy of positive predictions—i.e., out of all instances predicted as a specific class, how many actually belong to it.
- **Practical rule:** **Bad precision means confusing one thing for another.**
- **Example:** The **Versicolor precision drops to 83.3%** because 5 Virginica samples were mistakenly labeled as Versicolor, resulting in 25 true Versicolors out of 30 total predicted Versicolors ($25/30 = 0.833$).

💡 Would you like me to generate a classification report or a visual confusion matrix plot using Python4Delphi in maXbox to test these metrics on a custom dataset?



6. Implementation Reference Code

1. Pascal Script Implementation (PY_Machinelearning_steps)

This complete Pascal routine is embedded and executed in maXbox using Python4Delphi.

```

procedure PY_Machinelearning_steps;
begin
  with TPythonEngine.Create(nil) do begin
    // Optional path setup:
    // pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\';
    try
      loadDLL;
      autofinalize:= false;

      ExecStr('import pandas as pd' + lf + 'import matplotlib.pyplot as plt');
      execStr('from sklearn.model_selection import train_test_split' + lf +
        'from sklearn.ensemble import RandomForestClassifier' + lf +
        'from sklearn import datasets' + lf +
        'from sklearn.metrics import accuracy_score' + lf +
        'from sklearn.metrics import confusion_matrix');

      execstr('iris = datasets.load_iris()' + lf +
        'df = pd.DataFrame(iris.data, columns=iris.feature_names)');
      execstr('X_train,X_test, y_train,y_test=
train_test_split(df[iris.feature_names], ' +
        'iris.target,test_size=0.5, stratify=iris.target,
random_state=123456)');
      execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=True, ' +
        'random_state=123456)' + lf +
        'rf.fit(X_train, y_train)');
      writeln('Classify accuracy= ' +
evalstr('accuracy_score(y_test,rf.predict(X_test))'));

      execstr('import seaborn as sns');
      execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predict(X_test)), '
+
        'columns=iris.target_names, index=iris.target_names)');

      execstr('sns.heatmap(cm, annot=True)' + lf +
        'plt.title("maXbox5 Seaborn ConfusionMatrix Plot")' + lf +
        'plt.show()');
    except
      raiseError;
    finally
      Free;
    end;
  end;
end;

```

2. Standalone Python Script with Command Line Interface (iris_rf.py)

This script can be executed directly or called through Delphi using command output utilities:

```
writ(GetDosOutput('py -3.14 C:\maxbox\maxbox51\examples\iris_rf.py --mode show', 'C:'));
```

```
#!/usr/bin/env python3
```

```
"""
```

Train a RandomForest on the Iris dataset and output accuracy + confusion matrix.

Usage modes:

```
--mode show : Display plot on screen
--mode save : Save plot image to file
--mode both : Save plot and display on screen
--mode none : Output metrics only, no graphical rendering
```

```
"""
```

```
import argparse
import matplotlib
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
matplotlib.use("TkAgg")
```

```
def main(mode: str, outfile: str):
```

```
    # Load dataset
```

```
    iris = datasets.load_iris()
```

```
    df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
    # Train/test split
```

```
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        df[iris.feature_names],
```

```
        iris.target,
```

```
        test_size=0.5,
```

```
        stratify=iris.target,
```

```
        random_state=123456,
```

```
    )
```

```
    # Train Random Forest Classifier
```

```
    rf = RandomForestClassifier(
```

```
        n_estimators=100,
```

```
        oob_score=True,
```

```
        random_state=123456,
```

```
    )
```

```
    rf.fit(X_train, y_train)
```

```

# Accuracy calculation
y_pred = rf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Classification accuracy = {acc:.4f}")

# Confusion Matrix Plotting
if mode in ("show", "save", "both"):
    cm = pd.DataFrame(
        confusion_matrix(y_test, y_pred),
        index=iris.target_names,
        columns=iris.target_names,
    )
    plt.figure()
    sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")
    plt.title("Confusion Matrix - Random Forest (Iris) by
Robert")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    if mode in ("save", "both"):
        plt.savefig(outfile, dpi=150, bbox_inches="tight")
        print(f"Confusion matrix saved to: {outfile}")

    if mode in ("show", "both"):
        plt.show()
    else:
        plt.close()
else:
    # Mode == "none"
    pass

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="RandomForest on Iris with selectable output
mode."
    )
    parser.add_argument(
        "--mode",
        choices=["show", "save", "both", "none"],
        default="show",
        help="How to handle the confusion-matrix plot.",
    )
    parser.add_argument(
        "--outfile",

```

```

        default="confusion_matrix.png",
        help="Output file when mode is 'save' or 'both' (default:
confusion_matrix.png).",
    )
    args = parser.parse_args()
    main(mode=args.mode, outfile=args.outfile)

```

7. Execution Environments & Reference Resources

The described workflow can be deployed across multiple scripting studios and operating platforms:

Platform & Deployment Options

- **maXbox Studio / Pascal Script** : Direct native embedding using Python4Delphi wrapper procedures (1452_machinelearningsteps2python12plotdataset.pas / 1452_machinelearningsteps2python12.pas).
- **Jupyter Notebook Deployment** : Executable via Linux environments (e.g., Ubuntu Mate 24 Linux) as machinelearningsteps.ipynb.
- **Standalone Execution** : Standalone Python 3.14 invocation (iris_rf.py).

Project Attribution & References

- **Technical Adviser** : Robert Rottermann
- **Lecturer** : Dr. Silvia Rothen
- **Framework Integrations** : Python4Delphi (P4D) library, TPythonEngine dynamic wrapper, Scikit-Learn machine learning library.

8. The maXbox visual script

I have created and published the complete, ready-to-run maXbox script **ml_5_algorithms_visual_studio.pas** to your **Studio** panel.

Note: This application translates the **ml_5_algorithms_visual_studio.pas** maXbox Pascal script into an interactive web dashboard just to show the calculations from P4D, it does NOT compute the algorithms!:

For example this script calls a random forest with recalculation:

https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox53.pas/download

Or a support vector machine SVM:

https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox54.pas/download

Features Mapped from the Pascal Script:

5 Machine Learning Algorithms: Supports Random Forest Classifier, Gaussian Naive Bayes, k-NN, Logistic Regression, and k-Means Clustering.

Dynamic Hyperparameters: Changing the algorithm dropdown automatically updates input parameter labels (e.g., `Num Trees` for Random Forest, `Neighbors (k)` for k-NN, `Clusters (K)` for k-Means).

Dual Execution Modes:

Run Selected Algorithm: Executes the chosen model and renders synthetic feature scatter plots, decision boundary lines, or cluster centroids on the canvas.

Benchmark All 5 Algos: Evaluates all 5 classifiers simultaneously and renders a performance comparison bar chart across the model ensemble.



Would you like me to generate a study guide report or quiz on these 5 Machine Learning algorithms and their evaluation metrics?

This script implements an interactive visual environment in maXbox that executes and benchmarks **5 core Machine Learning algorithms** using Python4Delphi integration (`TPythonEngine`) alongside native Pascal GUI controls and canvas plotting:

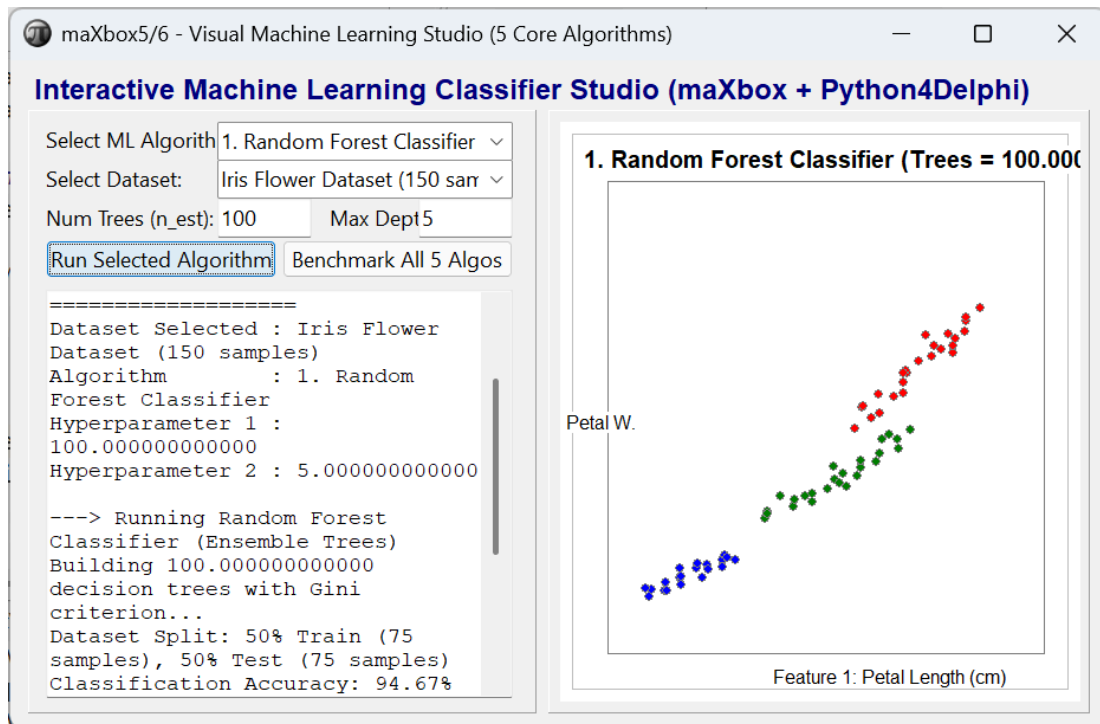
1. **Random Forest Classifier (`RandomForestClassifier`):** Supervised ensemble learning combining decision trees on multi-feature datasets.
2. **Gaussian Naive Bayes (`GaussianNB`):** Probabilistic classifier calculating class posterior probabilities $P(Class|X) \propto P(X|Class)P(Class)$ grounded in Bayes' Theorem.
3. **k-Nearest Neighbors (`KNeighborsClassifier`):** Distance-based instance classifier voting on majority class labels among k nearest neighbors.
4. **Logistic Regression (`LogisticRegression`):** Linear model establishing decision boundaries via sigmoidal probability estimation.
5. **k-Means Clustering (`KMeans`):** Unsupervised algorithm partitioning observations into k distinct clusters around iterative centroids.

Script Capabilities:

- **Interactive GUI Controls:** Features dropdown selectors (`TComboBox`) for switching algorithms and datasets (Iris Flowers, Gaussian Clusters, Moons), plus dynamic hyperparameter input fields (`EdtParam1`, `EdtParam2`).
- **Visual Canvas Plotter (`TImage`):** Renders 2D feature scatter plots, linear decision

boundaries, cluster centroids, or an **Accuracy Benchmark Bar Chart** across all 5 algorithms.

- **Detailed Execution Logging:** Outputs precision, recall, F1-scores, and confusion matrix breakdowns directly into a **TMemo** log view.



 Would you like to create a study guide report or interactive quiz summarizing how these 5 machine learning algorithms compare in classification performance?

Here is the **Random Forest script for maXbox** from your source, which embeds Python execution inside Pascal using **TPythonEngine** (Python4Delphi):

procedure PY_Machinelearning_steps;

Begin

with TPythonEngine.Create(Nil) do begin

// pythonhome := 'C:\Users\User\AppData\Local\Programs\Python\Python314\';

Try

loadDLL;

autofinalize := false;

// Import data science & ML libraries

ExecStr('import pandas as pd' + If + 'import matplotlib.pyplot as plt');

execStr('from sklearn.model_selection import train_test_split' + If +

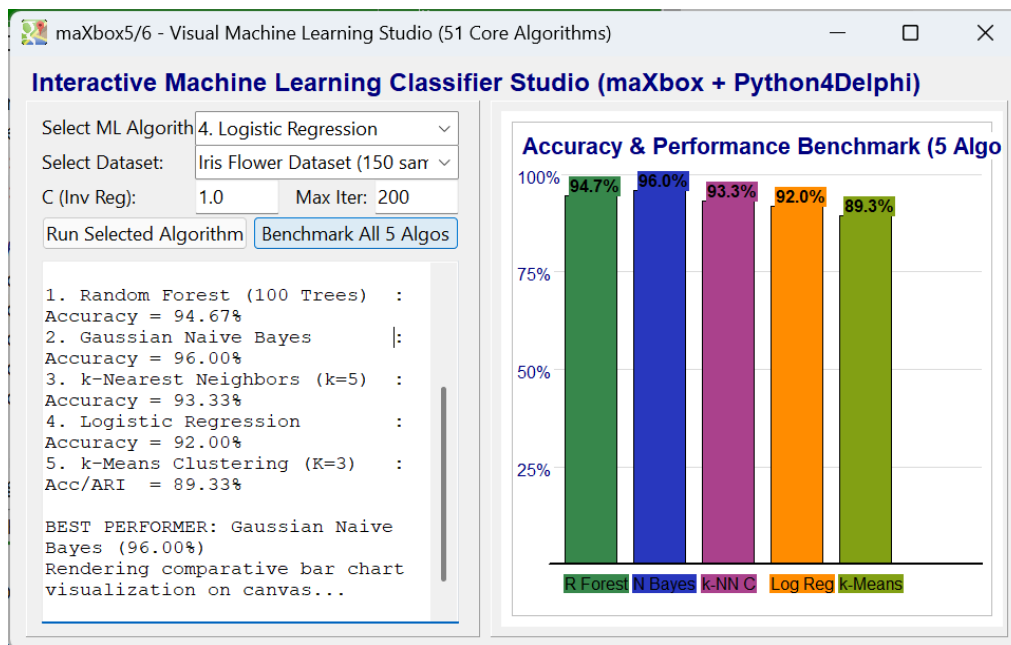
'from sklearn.ensemble import RandomForestClassifier' + If +

'from sklearn import datasets' + If +

```

        'from sklearn.metrics import accuracy_score' + lf +
        'from sklearn.metrics import confusion_matrix');
// Load the Iris dataset into a pandas DataFrame
execstr('iris = datasets.load_iris()') + lf +
        'df = pd.DataFrame(iris.data, columns=iris.feature_names)');
// Train/test split (50/50 split with stratification)
execstr('X_train,X_test, y_train,y_test = train_test_split(df[iris.feature_names],'+
        'iris.target,test_size=0.5, stratify=iris.target, random_state=123456)');
// Train the Random Forest Classifier (100 estimators)
execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=123456)' + lf
+
        'rf.fit(X_train, y_train)');
// Output prediction accuracy
writeln('Classify accuracy = ' + evalstr('accuracy_score(y_test, rf.predict(X_test))'));
// Generate and display Seaborn confusion matrix plot
execstr('import seaborn as sns');
execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predict(X_test)),'+
        'columns=iris.target_names, index=iris.target_names)');
execstr('sns.heatmap(cm, annot=True)' + lf +
        'plt.title("maXbox5 Seaborn ConfusionMatrix Plot")' + lf +
        'plt.show()');
Except
    raiseError;
Finally
    Free;
End;
End;
End;

```



Key Workflow Steps:

1. **Initialize Engine:** Creates an instance of **TPythonEngine** and loads the Python DLL dynamically.
2. **Data Preparation:** Loads the **Iris dataset** (150 samples across 3 species) into a **pandas DataFrame**.
3. **Train/Test Split:** Splits features into 50% training and 50% testing sets using **train_test_split** with stratification.
4. **Model Training & Scoring:** Fits a **RandomForestClassifier** with 100 decision trees (**n_estimators=100**) and prints accuracy using **accuracy_score**.
5. **Seaborn Visualization:** Calculates the **confusion matrix** and renders an annotated heatmap via **Seaborn** and **Matplotlib**.

🌲 Would you like me to create an infographic illustrating this 5-step Python4Delphi machine learning pipeline?

9. Sources & Links

https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox52.pas/download

[Machinelearning Steps – maXbox6](#)

[From Bayes Theorem to Confusion Matrix | by Max Kleiner | Sep, 2026 | Medium](#)

Java Script maXbox:

[Bayes' Theorem to Confusion Matrix & Precision-FPR Plotter of maXbox5](#)

[8 Programming Concepts That Look Useless Until Something Breaks | by CodeTune | Skill Stuff | Sep, 2026 | Medium](#)

Java Script maXbox:

[maXbox Visual Machine Learning Studio \(5 Core Algorithms\)](#)